Ders 27 Genişletilmiş Notu: Canlı C Kodlama: Min/Max ve Kısmi Sıralama ile Medyan

Ders 27 Genişletilmiş Notu: Canlı C Kodlama: Min/Max ve Kısmi Sıralama ile Medyan

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Projeden Gelen Teknik Sorular
- 2. Canlı Kodlama 1: En Küçük ve En Büyük Değer
- 3. Formatlı Yazdırma
- 4. Değer Yerine Yer Bulmak
- 5. Medyan Nedir, Nasıl Bulunur?
- 6. Sıralamayı Ortada Kesmek ve Kaba Kuvvet
- 7. Medyan Neden Önemli?
- 8. Canlı Kodlama 2: Karakter Dizisinin Medyanı
- 9. Medyan Sınırını Bulmak

1. Projeden Gelen Teknik Sorular

Hoca bu dersin kod yazmaya ayrıldığını, ama önce öğrencilerin dönem projesiyle ilgili sorularını alacağını söyledi. Bu soruların C diliyle ilgili olan kısımları aşağıdadır.

1.1 Yerel ve Global Değişkenler

Soru: Birden fazla fonksiyon kullandığımızda değişkenleri nerede tanımlamalıyız? Fonksiyonların tepesinde mi, yoksa yalnızca `main` içinde mi?

Cevap: Değişken tanımında genelde iki tip vardır: lokal (yerel) ve global. Ana fonksiyonun ya da bir fonksiyonun dışında tanımlanan değişkenlere global değişken denir ve hoca bu çalışmada bunların kullanılmamasını istedi. Bütün değişken tanımları bir fonksiyonun ya da ana fonksiyonun içinde yapılmalıdır.

Tanımladığınız her değişken yalnızca tanımlandığı yerde kullanılabilir. `x` adlı bir fonksiyonda tanımlanan değişken sadece orada geçerlidir. Ana fonksiyonda tanımlananlar da sadece ana fonksiyonda geçerlidir. Bu yüzden, başka bir fonksiyonun ihtiyaç duyduğu değişkeni o fonksiyona göndermek gerekir.

Değişken bilgisi fonksiyondan fonksiyona parametre olarak iki şekilde aktarılır:

- Değişkenin adres bilgisi gönderilir.
- Değişkenin değerinin bir kopyası gönderilir.

Bazen de fonksiyondan değer döndürmek gerekir. Bir fonksiyondan yalnızca bir değer döndürülebilir.

Öğrenci, program büyüdükçe kafasının karıştığını ve bu yüzden fonksiyona başvurmak istediğini söyledi. Hoca bu görüşe katıldı, ama fonksiyon konusuna geçebilmek için C'nin başka

önemli noktalarını da anlatmak gerektiğini, dersin süresinin buna yetmediğini belirtti. Kendi imkânlarıyla öğrenen kullanabilir.

Hoca, C'nin derste hızlı geçildiği eleştirisine de değindi. Bu eleştiride haklılık payı olduğunu söyledi. Ancak dersin beklentilerinden biri, gösterilen C tekniğini kendi çalışma saatinde kullanmak, derste çizilen algoritmaları koda dökerek pratik yapmaktır. C burada yalnızca temel yapıları (`while`, `if`, `for`, atama) öğrenmek için kullanılır. Bu dili daha ayrıntılı öğretmek bu dersin konusu değildir. Önümüzdeki yıllarda uygulama saatlerinde daha fazla kod yazılacağını da ekledi.

1.2 Tahtayı Ekranda Göstermek

Bir öğrenci, oyun tahtasını matriste tutarken gösterimin kötü durduğunu söyledi: Gidilen yol birlerle, piyonlar da 2'den 18'e kadar sayılarla görünüyordu. Bunun üzerine öğrenciler kendi yaklaşımlarını anlattı:

- **Tamsayıdan karaktere çevirmek:** Matrise tamsayılar doldurulur. Her yazdırmada bu değere göre ASCII tablosundan uygun karakter seçilir. Üç karakterli etiketler için üç karakterlik ayrı bir matris açılabilir.
- **Matriste pointer tutmak:** Hoca bunu şöyle açıkladı: Matrisin gözünde pointer saklarsanız, o pointer'ın adresindeki karakterleri ekrana yazdırabilirsiniz. Ama yazdırırken formatlı yazdırmak gerekir.
- **Arka planda indis tutmak:** Arka planda `int` tipinde bir matris tutulur. Her gözdeki değer, başka bir karakter matrisindeki bir etiketin indisidir.
- **Hücreyi genişletmek:** Tablo yazdırılırken her hücreye tek karakterlik yer ayırmak zorunlu değildir. Hücre genişliğine yazdırırken biz karar veririz. Örneğin her hücre iki karakter genişliğinde yazdırılabilir.

Hoca, bu çözümleri bulmanın işin bir parçası olduğunu, herkesin farklı bir çözümü olabileceğini söyledi.

2. Canlı Kodlama 1: En Küçük ve En Büyük Değer

Derste daha önce algoritması görülen problemin bu kez kodu yazıldı: Bir veri setindeki en küçük (min) ve en büyük (max) değeri bulan program. Hoca bunun çok sık kullanılan bir iş olduğunu söyledi.

2.1 Tanımlar ve Elemanların Okunması

Hoca önce eleman sayısını `int` olarak tanımladı, sonra bir an "bunu `float` yapalım" dedi ve hemen düzeltti: Eleman sayısı `float` olamaz. Eleman sayısı `int`, veri seti ise `float veriSeti[100]` olarak tanımlandı.

Eleman sayısı `%d` ile okunur. Elemanlar `for` döngüsüyle ve `%f` ile tek tek okunur:

```
for (i = 0; i < elemanSayisi; i++)
    scanf("%f", &veriSeti[i]);
```

> **Hocanın vurgusu:** En çok yapılan hatalardan biri budur. `scanf` içinde `veriSeti[n]` ya da `veriSeti[elemanSayisi]` yazanlar var. Bu doğru değildir. Her turda ilgili gözün, yani `i`. gözün okunması gerekir.

İstenirse kullanıcıya kaçıncı elemanı girdiği de gösterilebilir. Dizi 0'dan başladığı için ekrana `i + 1` yazdırılır:

```
printf("%d. elemani giriniz\n", i + 1);
```

2.2 Algoritma

Hatırlarsak, min ve max başlangıçta ilk elemana eşitlenir. Sonra dizinin geri kalanı gezilir. İlk eleman zaten atandığı için döngü `i = 1`'den başlar:

```
min = max = veriSeti[0];
for (i = 1; i < elemanSayisi; i++)
```

```
{
    if (veriSeti[i] < min)
        min = veriSeti[i];
    else if (veriSeti[i] > max)
        max = veriSeti[i];
}
```

İkinci kontrolün başına hoca bilerek yeni bir `if` değil `else` koydu. Çünkü eleman min'den küçükse, bir de büyük olup olmadığına bakmaya gerek yoktur. Küçük değilse büyük mü diye bakılır.

2.3 İlk Çalıştırma ve Hatalar

İlk derlemede `i`, `min` ve `max` tanımlanmadığı için hata alındı. Bunlar `int` olarak tanımlandı. Program şu girdiyle çalıştırıldı: eleman sayısı 4, elemanlar 1.2, −2.1, 0.4 ve 5.8. Çıkan sonuç yanlıştı ve hoca hatanın nedenini sordu.

Soru: Döngüyü 0'dan başlatmak gerekmez mi? Sonuçta 0. eleman da dizinin bir parçası.

Cevap: Hayır. 0. eleman zaten min ve max olarak atandı, yani başlangıçta onlar varsayılan min ve max oldu. Karşılaştırma bir sonraki elemandan başlar.

Hatayı sohbetten yazan bir öğrenci buldu: `min` ve `max` `int` tanımlanmıştı, oysa veri seti `float`'tır. Tanım `float` yapılınca aynı girdi için en küçük eleman −2.1, en büyük eleman 5.8 olarak yazdırıldı.

3. Formatlı Yazdırma

Hoca, yazdırmada `%f` yerine `%3.2f` yazılıp yazılamayacağını sordu. Bir öğrenci, noktadan sonraki kısmın kesirli basamakları belirlediğini söyledi. Program yeniden çalıştırıldı: 4 eleman, sırasıyla 1.2, −2.1, 0.67089 ve 5.98 ile başlayıp 7 ile devam eden bir sayı.

Sonuçta yalnızca virgülden sonraki iki basamak yazdırıldı. Sayı 98'den sonra 7 ile devam ettiği için 99'a yuvarlanarak yazdırıldı.

Soru: O zaman `3.2` sadece ne kadar gidebildiğini mi ifade ediyor?

Cevap: Hoca şöyle açıkladı: `%3`, noktadan önceki kısım için ne kadarlık bir alan kullanılacağını belirtir. Daha küçük sayılar girildiğinde de o kadarlık alanın işgal edilmesini istediğimizi formatta söylemiş oluruz. Noktadan sonraki kısım ise kesirli sayının ne kadar hassasiyette yazdırılacağını söyler.

> **Ders sonrası not:** Hoca 3'ün noktadan önceki kısım için ayrılan alan olduğunu söyledi. Doğrusu büyük olasılıkla şudur: Bu sayı, nokta ve kesirli kısım dahil bütün sayı için ayrılan en az genişliktir. Derste kullanılan −2.1 buna uyar: Ders sonrasında derlenen programda `%3.2f` ile başında boşluk olmadan `-2.10` yazıldı, çünkü bu yazım zaten 3 karakterden uzundur.

4. Değer Yerine Yer Bulmak

Şimdi soru değişti: En küçük ve en büyük değerleri değil, bunların veri setindeki yerini bulmak istesek, yani kaçıncı sayının en küçük, kaçıncı sayının en büyük olduğunu sorsak ne yapardık?

4.1 min ve max Olmadan Yazmak

Hoca `minYer` ve `maxYer` değişkenlerini ekledi ve ikisine 0 atadı, yani başlangıçta ilk gözün en küçük ve en büyük olduğunu belirledi. Bir öğrenci, `min` ve `max` değerlerini güncellemeye devam edip yanına `minYer = i` satırını eklemeyi önerdi.

Hoca bunun çalışacağını, ama daha kısa bir çözüm istediğini söyledi: `min` ve `max` değişkenleri hiç yokmuş gibi düşünün. Bu kısmı yorum satırı hâline getirdi. Öğrencilerle birlikte varılan sonuç şudur: `min` yerine `veriSeti[minYer]` yazılır. Yer bilindiğinde değere de doğrudan erişilmiş olur. Koşul doğruysa da yalnızca yer güncellenir.

```
minYer = maxYer = 0;
for (i = 1; i < elemanSayisi; i++)
{
    if (veriSeti[i] < veriSeti[minYer])
        minYer = i;
    else if (veriSeti[i] > veriSeti[maxYer])
        maxYer = i;
}
```

4.2 Kod Yazarken Çıkan Hatalar

Bu bölümde birkaç hata art arda çıktı. Hoca bazılarını bilerek yaptığını söyledi.

- **Yanlış karşılaştırma:** İkinci koşulda `veriSeti[maxYer]` yerine yanlış ifade yazılmıştı. Bir öğrenci uyardı.
- **Tanımlanmamış değişken:** `minYer` ve `maxYer` tanımlanmamıştı. Bunlar `int` kısmına eklendi.
- **Olmayan değişkeni yazdırmak:** `printf` hâlâ `min` ve `max` yazdırıyordu, oysa artık böyle değişkenler yoktu. Değer için `veriSeti[minYer]` ve `veriSeti[maxYer]` yazıldı. Yerleri de yazdırmak için `%d` eklendi. Her `%d`'nin karşısına ilgili değişken sırayla konuldu.
- **Büyük-küçük harf:** Değişkenin tanımında bir harf büyük, kullanımında küçük yazılmıştı.

  Hocanın vurgusu: Bu çok önemli bir nokta ve bilerek böyle yazıldı. C küçük-büyük harf ayrımını önemser. O yüzden değişkenin adı her yerde aynı yazılmalıdır.

  Ders sonrası not: Altyazıda hocanın "case sensitivity değildir" dediği duyuluyor, ama hemen ardından "küçük büyük harf ayrımını önemser" diyor. Doğrusu ikinci cümledir: C büyük olasılıkla büyük-küçük harfe duyarlıdır (case sensitive).

4.3 Kullanıcıya Yeri Doğru Göstermek

Program 4 elemanla çalıştırıldı: Bir sayı, 345.213, 0.0007 ve 5.9999. Hoca küsuratın atıldığını gördü: En küçük eleman 0, yeri 2; en büyük eleman 345, yeri 1 olarak yazdırıldı.

Dizi 0'dan başladığı için kullanıcı burada yanlış bilgi görür: 0.0007 aslında üçüncü sayı, 345.213 ikinci sayıdır. Bir öğrenci çözümü söyledi: Yazdırırken yere 1 eklenir (`minYer + 1`, `maxYer + 1`). Hoca, kullanıcının anlayacağı dilde ve daha doğru görünen çıktı için böyle düzenlemeler yapmak gerektiğini söyledi.

Hoca bu sırada derlemeyi de hatırlattı: Her değişiklikten sonra "run" tuşuna basıyor, arka planda ise derleme (compilation) çalışıyor. Kod derlenmezse değişiklikler programın çalışmasına yansımaz.

4.4 Son Çalıştırma ve Yuvarlama

Son olarak 5 elemanlı bir veri girildi: −22.22, −99.99, 5.99, 3.1 ve 0.01.

- En küçük eleman −99.99'dur ve ikinci sıradadır. −100 olarak yazdırıldı.
- En büyük eleman 5.99'dur ve üçüncü sıradadır. 6 olarak yazdırıldı.

Hoca yuvarlamayı şöyle açıkladı: Biri yukarı doğru (5.99'dan 6'ya), diğeri aşağı doğru (−99.99'dan −100'e) yuvarlandı. Dokuzlardan dolayı kesirli kısım sıfırlandı ve yuvarlama tam kısma yansıdı.

Son program şöyledir. `printf` metinleri altyazıdan kuruldu ve Türkçe karakter kullanılmadan yazıldı.

```
#include <stdio.h>

int main()
{
    int elemanSayisi, i, minYer, maxYer;
    float veriSeti[100];
```

```
    printf("Eleman sayisini giriniz\n");
    scanf("%d", &elemanSayisi);

    printf("Elemanlari giriniz\n");
    for (i = 0; i < elemanSayisi; i++)
        scanf("%f", &veriSeti[i]);

    minYer = maxYer = 0;
    for (i = 1; i < elemanSayisi; i++)
    {
        if (veriSeti[i] < veriSeti[minYer])
            minYer = i;
        else if (veriSeti[i] > veriSeti[maxYer])
            maxYer = i;
    }

    printf("Dizinin en kucuk elemani: %.0f, yeri: %d\n", veriSeti[minYer], minYer + 1);
    printf("Dizinin en buyuk elemani: %.0f, yeri: %d\n", veriSeti[maxYer], maxYer + 1);
    return 0;
}
```

Hoca bu işin N adımda yapıldığını söyledi: Bütün diziye bir kez bakılır.

5. Medyan Nedir, Nasıl Bulunur?

Hoca yeni bir soru sordu: Bir veri setinin medyanını bulmak isteseydiniz ne yapardınız? Bir öğrenci eleman sayısını ikiye bölmeyi önerdi. Hoca, bunun ne yapmak istediğini tam anlatmadığını söyledi ve önce tanımı hatırlattı.

Medyan (ortanca), küçükten büyüğe sıralı bir veri grubundaki en ortadaki elemandır. Sıralı bir diziye baştan sona baktığınızda en ortada bulunan eleman o dizinin medyanını gösterir.

Ara verilirken hoca, bir veri setinin medyanını bulan algoritmanın akış diyagramını çizmelerini istedi. Dizinin sıralı olduğu varsayılmayacaktı: Karışık bir veri grubunun medyanı isteniyordu.

5.1 Öğrencilerin Çözümleri

Aradan sonra bir öğrenci çözümünü anlattı: Önce diziyi sıralamak (kendisi insertion sort kullanmıştı, bubble sort ve merge sort da olabilir). Sonra eleman sayısının tek mi çift mi olduğuna bakmak. Tekse ortadaki elemanı, çiftse ortadaki iki elemanın toplamının yarısını yazdırmak.

Hoca çift sayıda eleman durumuna dikkat çekti. İki ortadaki elemanı toplayıp ikiye bölmek bir yaklaşımdır, ikisinden birini almak da bir yaklaşımdır. Teoride ortadaki iki eleman toplanıp ikiye bölündüğü için bu kesinlikle yanlış değildir. Ama bilgisayar bilimlerinde veri gruplarıyla çalışırken bu ayrıntıyı hesaba katmadan, tek mi çift mi diye bakmadan doğrudan ortadaki elemanı alarak da işlem yapılabilir.

Başka bir öğrenci aynı yöntemi farklı bir sıralamayla yaptığını söyledi. Bir diğeri ise önemli bir fark ekledi: Insertion sort kullanmış, ama sıralamayı yalnızca ortadaki elemana kadar sürdürmüş, daha ileri gitmemişti. Hocanın bu problemi derse getirme nedeni tam olarak buydu.

6. Sıralamayı Ortada Kesmek ve Kaba Kuvvet

> **Hocanın vurgusu:** Bu çok çok önemli bir ayrıntıdır. Medyanı bulmak için sıralama yapıyorsanız, sıralamanın yalnızca medyana kadar olan kısmını yapmanız gerekir. Medyandan sonrasını sıralamaya çalışmak gereksiz bir çaba, gereksiz bir iş yüküdür. Bir milyon tane veriniz varsa yalnızca 500 binini sıralamış olursunuz. Yüksek boyutlu veri setlerinde bu önem kazanır ve zaman kazandırır. Soruyu doğru çözmüş olsalar bile, tüm diziyi sıralayan ile ortasına kadar sıralayan aynı değerlendirilmez.
>
> **Ek bilgi (derste anlatılmadı):** Selection sort ile bir milyon elemanın ortasına kadar sıralamak, işi büyük olasılıkla yarıya değil yaklaşık dörtte üçüne indirir. Çünkü her

turda iç döngü kalan elemanların tamamına bakar ve ilk turlar en pahalı turlardır. Karşılaştırma sayısı tüm dizi için yaklaşık 5×10^{11}, ortaya kadar yaklaşık $3{,}75 \times 10^{11}$ çıkar. Yani yazılan elemanlar yarıya iner, ama karşılaştırma tasarrufu yüzde 25 civarındadır.

Ders sonrası not: Hoca, sıralamanın ortaya kadar yapılmasını genel olarak söyledi ve başka bir sıralama algoritması da kullanılabileceğini ekledi. Bu büyük olasılıkla her algoritma için geçerli değildir. Selection sort her turda kalanların en küçüğünü seçtiği için ilk yarıda gerçekten en küçükler toplanır. Insertion sort ise yalnızca o ana kadar gördüğü kısmı sıralar: Dizinin ilk yarısında durursa ikinci yarıya hiç bakmamış olur ve ortadaki eleman medyan olmayabilir. Ders sonrasında rastgele dizilerle yapılan python denemesinde selection sort hep doğru, yarıda kesilen insertion sort çoğu dizide yanlış sonuç verdi.

Hoca bu noktadan genel bir öneriye geçti. İlk akla gelen basit çözümler genelde kaba kuvvet (brute force) yaklaşımlarıdır. Bunların bir kısmı zaten olması gereken verimli çözümlerdir, bir kısmı ise değildir ve iyileştirilmesi gerekir. Önce algoritmanın doğru çalışması önemlidir, ama mümkünse ilk aklınıza gelenle yetinmemelisiniz.

Hocanın vurgusu: Size verilen problemin ne istediğini, girdinin ve çıktının ne olduğunu doğru tespit edip ona göre bir çözüm üretmeye çalışın.

7. Medyan Neden Önemli?

Hoca kodu yazmadan önce bir soru daha sordu: Medyan bulmanın ortalama bulmadan farkı nedir, neden önemlidir? Birkaç cevap geldi.

Soru: Ortalama doğrudan elde edilemez mi?

Cevap: Birden n'e kadar olan sayıların toplamı için formül vardır: $n(n+1)/2$. Ama genel olarak ortalama için bütün sayıları toplayıp n'e bölmek zorundasınız. Bu, medyanın neden kullanıldığı sorusuna cevap vermiyor.

Bir öğrenci, medyanın arama işinde kullanılabileceğini söyledi: Çok büyük bir dizide bir şey ararken ortadaki değere bakılır. Aranan büyükse sağ tarafa, küçükse sol tarafa bakılır ve böylece daha hızlı işlem yapılır. Hoca bunun uzun vadede kullanılabilecek bir yaklaşım olduğunu, ama sorduğu şeyin daha basit, istatistikteki kullanım amacı olduğunu söyledi.

Başka bir öğrenci, aritmetik ortalamanın veri setinde olmayan bir değer verebildiğini, medyanın ise veri setinde var olan bir değer verdiğini söyledi. Hoca bunun yanlış olmadığını, ama bir ek bilgi olduğunu belirtti.

Esas amaç şudur: Ortalama alırken veri grubunun orta noktasını ifade etmeyi hedefleriz. Ama standart sapma çok yüksekse ortalama bunu her zaman başaramaz. Hocanın ilk örneğinde min 0, max 6 ise orta nokta 3 görünür. Bu her koşulda veri grubunun ortasını temsil etmez.

İkinci örnek daha açıktır: Veride 0 ve 100 var, aradaki değerlerin hepsi 22-23 civarında. 0 ve 100'ü de ortalamaya katarsanız, ortalama 30-35 çıkabilir. Oysa veri grubunun oturduğu nokta 22-23'tür. Medyanda ise elemanları dizip en ortadakini alırsınız. Hoca bunu "standart sapmadan bağımsız bir veri" üretmek olarak tanımladı.

Ders sonrası not: Hocanın verdiği 30-35 aralığı ara değerlerin sayısına bağlıdır. Ara değerler 22.5 alındığında bu aralık büyük olasılıkla yalnızca 3 ila 5 ara değer için tutar. Ara değer sayısı arttıkça ortalama 22-23'e yaklaşır. Hocanın ana fikri bundan etkilenmez: Uç değerler ortalamayı çeker, medyanı çekmez.

Hocanın vurgusu: Verinin merkezini daha doğru ifade etmek için standart sapması yüksek veri gruplarında medyan bulmayı tercih ederiz.

Ek bilgi (derste anlatılmadı): Büyük olasılıkla burada belirleyici olan standart sapmanın kendisinden çok uç değerlerin ve çarpık dağılımın varlığıdır. Medyan uç değerlerin büyüklüğünden etkilenmez, çünkü yalnızca sıraya bakar. Buna istatistikte dayanıklılık (robustness) denir.

8. Canlı Kodlama 2: Karakter Dizisinin Medyanı

Hoca önceki programın iskeletini korudu. `min` ve `max` silindi, eleman sayısı bırakıldı, veri seti karakter tipine çevrildi ve okuma `%c` yapıldı. Sıralama için daha önce görülen selection sort, bubble sort ve insertion sort arasından selection sort seçildi.

8.1 Selection Sort İskeleti

Önce sıralama bütün dizi için yazıldı. Dış döngü `elemanSayisi - 1`'e kadar gider. Her turda `min = i` kabul edilir, `j` de `i + 1`'den başlayıp kalanları gezer. Kalanlar arasında daha küçük bir eleman bulunursa `min` güncellenir:

```
for (i = 0; i < elemanSayisi - 1; i++)
{
    min = i;
    for (j = i + 1; j < elemanSayisi; j++)
        if (veriSeti[j] < veriSeti[min])
            min = j;
    tmp = veriSeti[i];
    veriSeti[i] = veriSeti[min];
    veriSeti[min] = tmp;
}
```

Burada `min`, önceki programdaki gibi bir değer değil, en küçük elemanın yeridir.

8.2 Yer Değiştirmede Bilerek Yapılan Hata

Hoca iç döngüden çıkınca yer değiştirmeyi önce `tmp = veriSeti[min]` diye yazdı ve doğru olup olmadığını sordu. Bir öğrenci hatayı buldu: `tmp`'ye `veriSeti[i]` alınmalıdır.

Hoca bunun çok önemli bir hata olduğunu ve bilerek yazdığını söyledi. `tmp`'ye `veriSeti[min]` alınırsa `veriSeti[i]` saklanmamış olur. Doğru sıra şöyledir: `veriSeti[i]` saklanır, `i`. gözün üzerine `min`. gözdeki bilgi yazılır, sonra `min`. gözün üzerine saklanan `i` bilgisi yazılır.

Soru: İçteki `for` için küme parantezi gerekmez mi?

Cevap: Bu kullanımda bir yanlışlık yok. `for`'a ait tek bir iş var (`if`), `if`'e ait de tek bir iş var (`min = j`). Ama parantez kullanmak her zaman daha garantidir.

Soru: Eleman sayısının bir eksiğine kadar gidiyoruz, yarısına kadar gitmeyecek miydik?

Cevap: Evet, onu birazdan düzelteceğiz.

8.3 Enter Tuşu Sorunu: `scanf(" %c", ...)`

Medyanı `%c` ile yazdırmak için `printf`'e `veriSeti[elemanSayisi / 2]` yazıldı. Eksik `min`, `j` ve `tmp` tanımlandı (`tmp`, karakter tipinde olmalıdır). Eleman sayısı 5 girilip harfler yazılmaya başlandığında program daha harfler bitmeden sona erdi. Bir öğrenci çözümü söyledi: `%c`'nin soluna bir boşluk bırakılır.

```
scanf(" %c", &veriSeti[i]);
```

Hoca nedenini şöyle anlattı: Enter tuşuna basıldığında `scanf` onu da bir karakter olarak algılar. Yani verdiğiniz bilgiyle Enter'ı iki karakter gibi okur. Boşluk bırakıldığında Enter okunmamış olur.

Soru: Enter'a basınca `\n` gidiyor. Boşluk bırakmak neden `\n`'i de engelliyor?

Cevap: Hoca, `scanf`'in çalışma prensibinin boşluk görene kadar okumak olduğunu ve Enter'ın da bu kapsamda algılandığını söyledi. Bu durumun çok sempatik olmadığını ve anlatmasının da kolay olmadığını ekledi.

> **Ders sonrası not:** Hocanın açıklaması soruyu tam karşılamadı. Doğrusu büyük olasılıkla şudur: Format metnindeki bir boşluk, girdideki boşluk türü bütün karakterleri (boşluk, sekme, yeni satır) okuyup atlar. Bu yüzden `\n` de atlanır. Ders sonra-

sında derlenen programda boşluksuz `%c` kullanıldığında ilk okunan karakter yeni satır oldu.

Soru: Boşluk bırakmak yerine başka bir çözüm var mı?

Cevap: Karakterleri tek tek almayıp karakter grubu olarak `%s` ile hepsini birlikte okuyabilirsiniz.

Sohbette, `scanf`'in karakterlerle kullanılmasının tavsiye edilmediği ve neyle okunması gerektiği soruldu. Hoca, `scanf`'in sık kullanılan bir fonksiyon olduğunu, ama dışarıdan yapılacak ataklara açık olduğunu daha önce söylediklerini hatırlattı. Sohbetten gelen `fscanf` cevabının doğru olduğunu, ama `fscanf` akıştan (stream) okuduğu için dosya konusuna girdiğini söyledi. Bu ayrıntılar C programlama dersinde görülecektir.

9. Medyan Sınırını Bulmak

Program çalıştıktan sonra asıl konuya gelindi: Sıralama nereye kadar yapılmalı ve hangi göz yazdırılmalı? Hoca bunu denemelerle, öğrencilerle birlikte buldu. n eleman sayısı, `medyanSinir` ise birazdan tanımlanacak sınırdır.

9.1 Tüm Diziyi Sıralamak: Doğru Ama Gereksiz

Girdi: `a c z k m` ($n = 5$). Dış döngü `elemanSayisi - 1`'e kadar gidince dizi `a c k m z` olur. `5 / 2` = 2 olduğu için `veriSeti[2]`, yani `k` yazdırılır. Sonuç doğrudur, ama dizinin tamamı sıralanmıştır (ara adım eklendi).

9.2 `i < elemanSayisi / 2`: Orta Göz Sıralanmıyor

Dış döngü `elemanSayisi / 2`'ye kadar çalıştırıldı. Aynı girdiyle bu kez `z` yazdırıldı.

Soru: Neden böyle oldu?

Cevap: Bir öğrenci açıkladı: Eleman sayısı 5 iken ikiye bölününce tam sayı bölmesi 2 verir. Döngü `i = 0` ve `i = 1` için döner. Dizi `a c z k m` olarak kalır ve 2. göz hiç sıralamaya girmemiş olur. Orada duran `z` yazdırılır.

9.3 `(elemanSayisi + 1) / 2`: Beş Elemanda Doğru

Sınır `(elemanSayisi + 1) / 2` yapıldı. $n = 5$ için $6/2 = 3$ olur, döngü `i = 0, 1, 2` için döner. Dizi `a c k z m` olur, `veriSeti[2]` yine `k`'dır ve beklenen cevap alınır. Dizinin geri kalanı sıralanmamıştır, ama buna gerek yoktur (ara adım eklendi).

9.4 Sınırı Döngü Koşuluna Yazmamak

Hoca kodun doğru çalışmasına rağmen burada istenmeyen bir şey olduğunu söyledi: `(elemanSayisi + 1) / 2` ifadesinin döngü koşulunda yazılması. Öğrencilerin cevabı, kodu dışarıdan okuyan birinin bu ayrıntıyı zor yakalaması ve çift sayıda elemanda fazladan iş yapılmasıydı. Hoca ikisini de aradığı cevap olarak görmedi. Aradığı cevabı başka bir öğrenci verdi: Her turda fazladan işlem yapılıyor.

Hoca, iyi bir derleyicinin bu işlemi her turda yapmayacağını, ama kötü bir derleyicinin her turda eleman sayısına 1 ekleyip ikiye böleceğini söyledi. Bu yüzden sınır bir kez hesaplanıp bir değişkende tutulur:

```
medyanSinir = (elemanSayisi + 1) / 2;
for (i = 0; i < medyanSinir; i++)
```

9.5 Altı Elemanda Sorun

Hoca aynı kodu çift sayıda elemanla denedi. Girdi: `a c e z g m` ($n = 6$). `z` çıktı.

Adım adım bakıldığında $(6 + 1)/2 = 3$ (tam sayı bölmesi), yani döngü `i = 0, 1, 2` için döner ve ilk üç göz `a c e` olur. Oysa `printf` hâlâ `veriSeti[elemanSayisi / 2]`, yani `veriSeti[3]` yazdırmaktadır. Bu göz sıralamaya girmemiştir ve orada `z` durmaktadır (ara adım eklendi).

Bir öğrenci `printf`'in değiştirilmesini önerdi. `veriSeti[medyanSinir]` yazıldı ve denenen girdide sonuç düzelmiş göründü (`g`).

Bir sonraki soru dış döngünün koşuluydu: `i < medyanSinir` mı, `i <= medyanSinir` mı?

Soru: `i < medyanSinir` mı olmalı, `i <= medyanSinir` mı?

Cevap: Bir öğrenci, döngü 0'dan başladığı için küçük olması gerektiğini, eşit olsaydı yanlış olmayacağını, ama daha fazla işlem yapacağını söyledi. Hoca bunun çok sınırda bir soru olduğunu, bu soruları farklı bakış açıları oluşsun diye gündeme getirdiğini söyledi.

9.6 `%d` ve `%c`: ASCII Karşılığı

Hoca bu sırada harflerin sayısal karşılıklarını görmek için `%c` yerine `%d` yazdı. Girdi: `a c z e m g`. Çıktı 122 oldu.

122 hangi harfe denk geliyor? Küçük `a` 97'den başlar ve `z` 122'ye denk gelir. Hoca 97'yi ezberlemek gerektiğini söyledi.

> **Hocanın vurgusu:** C'de `%d` ile yazdırırsanız ASCII karşılığını, `%c` ile yazdırırsanız karakter karşılığını alırsınız.

Burada ilginç bir şey görüldü: 6 elemanda medyan olarak `z` gelmişti. `z` dizinin en büyük elemanıdır ve medyan olamaz. Demek ki hâlâ bir hata vardı. Adım adım (ara adım eklendi): `medyanSinir` 3 olduğu için `i = 0, 1, 2` döner ve dizi `a c e z m g` olur. `veriSeti[3]` sıralanmamıştır ve orada `z` vardır.

9.7 Asıl Hata: Yazdırılan Göz Sıralanmış Olmalı

İki görüş ortaya çıktı. Bir öğrenci diziyi sonuna kadar sıralamak gerektiğini düşündü. Başka bir öğrenci sorunun az önceki eşittir tartışmasıyla ilgili olduğunu söyledi: Çift sayıda elemanda sınırdaki eleman sıralamaya dahil edilmiyordu.

Hoca ikincisini denedi. Döngü `i <= medyanSinir` yapıldı, `i = 0, 1, 2, 3` için döndü ve dizi `a c e g m z` oldu. `veriSeti[3]`, yani `g` yazdırıldı (ara adım eklendi). `medyanSinir - 1` gözündeki `e` de doğru yerdeydi. Demek ki sıralamanın kendisinde bir sorun yoktu.

> **Hocanın vurgusu:** Sıralamada o medyan sınırı değerinin de sıralamaya girmiş olması lazım. Oradaki elemanı sıralamaya sokmazsanız, ortadaki gözde ne kalırsa onu ekrana yazdırırsınız.

Hoca, buradaki sıralamanın selection sort olduğunu ve veri setinin ortasına kadar yapıldığını, yerine farklı bir sıralama da kullanılabileceğini söyledi (bkz. Bölüm 6'daki ders sonrası not).

9.8 Beş Elemanla Son Kontrol: `medyanSinir - 1`

Soru: Yazdırırken `medyanSinir - 1` yazsaydık da aynı sonucu alır mıydık?

Cevap: Hoca önce önceki kodun 5 elemanda doğru çalışmadığını göstermek istedi. Bu sınamada ortaya `c` gelmesi gereken bir dizide, yani sıralı hâli `a b c e z` olan bir dizide, `veriSeti[medyanSinir]` yazdırıldığı için `e` çıktı. Çünkü $n = 5$ için `medyanSinir` 3'tür ve 3. göz dördüncü elemandır.

`veriSeti[medyanSinir - 1]` yazıldı. Girdi `a b z c e` için `c` bulundu. Girdi 6 elemanlı olunca da üçüncü eleman verildi.

Hoca çift sayıda elemanı şöyle yorumladı: Tek mi çift mi kontrolü yapmadan çözdüğümüz için 6 elemanda sonuç üçüncü elemandır. Dördüncü eleman da verilebilirdi, sakıncası yok. Kontrol yapılsaydı üçüncü ile dördüncüyü toplayıp ikiye bölmek de mümkün olurdu.

> **Ders sonrası not:** Hoca 6 elemanlı denemeden sonra esas sorunun `<` kontrolü olduğunu söylemişti. Bu ara bir tespittir ve büyük olasılıkla son durum şudur: Beş elemanlı sınamanın gösterdiği gibi `i <= medyanSinir` ile `veriSeti[medyanSinir]` birlikte doğru sonuç vermez. Doğru olan, yazdırılan gözün sıralamaya girmesi ve bu gözün medyan gözü olmasıdır. `i < medyanSinir` ve `veriSeti[medyanSinir - 1]` bunu

fazladan tur olmadan sağlar. Hoca altyazıda döngü koşulunu son hâlinde açıkça söylemiyor.

Karakterlerde de bir fark vardır:

> **Hocanın vurgusu:** Sayısal karşılık alabilirsiniz, ama sıraladığınızda karakterlerde medyan bulurken ortadaki ikisini toplayıp ikiye bölmek doğru bir sonuç vermez. Bu yüzden çift sayıda elemanda ortadaki elemanlardan biri alınır.

9.9 Son Program

Aşağıdaki program, derste varılan kararlarla kuruldu: `i < medyanSinir`, `veriSeti[medyanSinir - 1]` ve `" %c"`. Girdi `a b z c e` için `c`, girdi `a e z c g m` için `e` verir.

```c
#include <stdio.h>

int main()
{
    int elemanSayisi, i, j, min, medyanSinir;
    char veriSeti[100], tmp;

    printf("Eleman sayisini giriniz\n");
    scanf("%d", &elemanSayisi);

    printf("Elemanlari giriniz\n");
    for (i = 0; i < elemanSayisi; i++)
        scanf(" %c", &veriSeti[i]);

    medyanSinir = (elemanSayisi + 1) / 2;
    for (i = 0; i < medyanSinir; i++)
    {
        min = i;
        for (j = i + 1; j < elemanSayisi; j++)
            if (veriSeti[j] < veriSeti[min])
                min = j;
        tmp = veriSeti[i];
        veriSeti[i] = veriSeti[min];
        veriSeti[min] = tmp;
    }

    printf("Veri setinin medyani: %c\n", veriSeti[medyanSinir - 1]);
    return 0;
}
```

Dersin Toparlaması

Bu derste birlikte iki program yazıldı. İlkinde bir veri setinin en küçük ve en büyük elemanı bulundu. Değer bulmak ile yer bulmak arasındaki fark görüldü: Yer bilindiğinde değere de erişilir, bu yüzden `min` ve `max` değişkenlerine gerek kalmaz. Yer kullanıcıya gösterilirken 1 eklenir. `float` ve `char` tipleriyle çalışılırken formatlı yazdırma, `%d` ile `%c` farkı ve `%c`'nin Enter ile ilişkisi görüldü.

İkincisinde medyanın ne olduğu, neden önemli olduğu ve bulunurken nelere dikkat edileceği konuşuldu. Ana fikir, problemin ne istediğini doğru tespit edip gereksiz işi yapmamaktır: Medyan için sıralama ortaya kadar yapılır. Kod yazarken hata çıktığında nasıl düzeltileceği de birlikte görüldü. Hoca, bir sonraki derste ağ ve internetle ilgili kalan birkaç bilgiyi paylaşacağını ve ardından ikili arama (binary search) algoritmasını çözeceğini söyledi.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.